

CURSO DE JAVA

CAPÍTULO 4

Variables en Java

Desde principiante hasta nivel práctico

Enfoque: Desarrollo de Java#

Capítulo técnico, educativo y práctico para comprender, practicar y aplicar variables en Java.

Objetivo del capítulo

Aprenderás qué es una variable, por qué Java necesita variables, cómo se declara, inicializa, modifica y utiliza, y cómo elegir tipos de datos adecuados. También comprenderás el alcance de las variables y su papel en sistemas reales.

Al finalizar este capítulo serás capaz de:

- Explicar qué es una variable y qué problema resuelve.
- Declarar e inicializar variables correctamente.
- Identificar y utilizar los tipos primitivos principales y String.
- Elegir un tipo de dato apropiado según la información.
- Modificar y reutilizar valores durante la ejecución.
- Distinguir declaración, inicialización, asignación y uso.
- Comprender el modelo básico de variable, valor, tipo y alcance.
- Reconocer variables locales, parámetros y atributos.
- Detectar errores de tipos, inicialización y nombres.
- Aplicar variables en máquinas, sensores, inventarios y sistemas empresariales.

Introducción al tema

Imagina una caja etiquetada en un almacén. La etiqueta dice “temperaturaMotor” y dentro hay un valor como 72.5. Una variable cumple una función parecida: proporciona un nombre mediante el cual el programa puede trabajar con un dato. La diferencia es que Java aplica reglas precisas sobre el tipo de dato y el lugar donde existe la variable.

Las variables existen porque los programas necesitan trabajar con información que cambia o que debe reutilizarse. Una temperatura puede cambiar, una producción puede aumentar, un cliente puede tener un nombre y una máquina puede pasar de activa a inactiva. Sin variables, los programas serían rígidos y muy difíciles de construir.

En proyectos reales aparecen en máquinas industriales, sensores, PLC, inventarios, empleados, clientes, robots, drones, videojuegos y aplicaciones empresariales. Se conectan con salidas, operadores, decisiones, ciclos, métodos y programación orientada a objetos.

1. ¿Qué es una variable?

Qué es: un elemento nombrado del programa que representa un valor y tiene un tipo y un alcance determinados.

Por qué se utiliza: permite guardar, identificar, reutilizar y modificar información.

Cuándo se utiliza: cuando un programa necesita trabajar con datos que serán leídos, calculados, comparados o actualizados.

Cómo funciona: se declara el tipo y nombre, se asigna un valor y después el programa puede leerlo o cambiarlo cuando las reglas del tipo y alcance lo permiten.

Sintaxis

```
tipo nombreVariable = valor;
```

Ejemplo básico

```
int edad = 35;
System.out.println(edad);
```

int es el tipo; **edad** es el nombre; **=** asigna; **35** es el valor inicial. La segunda línea utiliza la variable.

Ejemplo práctico industrial

```
double temperatura = 72.5;
System.out.println("Temperatura: " + temperatura + " °C");
```

La variable representa una medida. **double** permite un valor decimal y puede actualizarse cuando llegue una nueva lectura.

2. Declaración, inicialización y asignación

Declarar significa indicar tipo y nombre. Inicializar significa proporcionar el primer valor. Asignar significa colocar un valor en la variable; la inicialización es una asignación inicial.

```
int velocidad; // declaración
velocidad = 1500; // asignación
int piezas = 500; // declaración + inicialización
```

Una variable local debe recibir un valor antes de utilizarse. Java comprueba esta regla al compilar.

```
int temperatura;
System.out.println(temperatura); // Error de compilación
```

3. Tipos de datos en Java

Un tipo de dato indica qué clase de valor representa una variable y qué operaciones son válidas. Java tiene tipado estático, por lo que el compilador verifica muchas incompatibilidades antes de ejecutar.

Tipo	Representa	Ejemplo
byte	Enteros pequeños	byte nivel = 10;
short	Enteros pequeños/medios	short piezas = 1200;
int	Enteros comunes	int edad = 35;
long	Enteros grandes	long total = 5000000000L;
float	Decimal de precisión simple	float voltaje = 24.5f;
double	Decimal general	double temperatura = 72.5;
char	Un carácter	char estado = 'A';
boolean	Verdadero/falso	boolean activo = true;
String	Texto	String nombre = "Luis";

Los ocho primeros de la tabla son tipos primitivos. **String** es una clase de Java para representar texto. Esta diferencia será importante al estudiar objetos y referencias.

4. Tipos primitivos explicados desde cero

Enteros: byte, short, int y long

Representan números sin parte decimal. En la práctica, **int** suele ser la opción general para cantidades enteras; **long** se utiliza cuando el rango necesario es mayor.

```
int piezas = 1500;
long produccionTotal = 5000000000L;
```

El sufijo L indica que el literal se trata como long.

Decimales: float y double

```
double temperatura = 72.5;
float voltaje = 24.5f;
```

double es habitual para cálculos decimales generales. float utiliza menor precisión y normalmente necesita el sufijo f. Los números de punto flotante pueden representar algunos valores de forma aproximada.

boolean

```
boolean maquinaActiva = true;
boolean alarma = false;
```

Solo puede ser true o false y resulta fundamental para estados y decisiones.

char y String

```
char letra = 'A';
String maquina = "Torno CNC";
```

char representa un carácter y usa comillas simples. String representa texto y usa comillas dobles.

5. Variables y memoria: modelo mental

Para comenzar, piensa en la memoria como el espacio donde el programa trabaja con datos. El modelo útil para el estudiante es: nombre + tipo + valor + alcance. Los detalles internos de la JVM son más complejos y se estudiarán posteriormente.

Elemento	Pregunta que responde
Nombre	¿Cómo identifico el dato en el código?
Tipo	¿Qué clase de valor representa?
Valor	¿Qué contiene actualmente?
Alcance	¿Desde qué parte puedo utilizarlo?
Duración	¿Durante qué contexto existe?

Flujo conceptual: declaración → variable disponible → asignación → lectura/uso → posible nueva asignación.

6. Reasignación y cambio de valores

Una variable que no es final puede recibir otro valor compatible con su tipo.

```
int velocidad = 1500;
System.out.println(velocidad);

velocidad = 1600;
System.out.println(velocidad);
```

Primero contiene 1500 y después 1600. La variable conserva su nombre y tipo, pero cambia su valor.

Ejemplo industrial

```
double temperatura = 70.0;
temperatura = 74.5;
temperatura = 76.2;
```

Estos cambios pueden representar lecturas sucesivas de una máquina. En un proyecto inicial los valores pueden ser simulados; posteriormente podrían venir de entradas, archivos, sensores o servicios.

7. Constantes con final

`final` impide reasignar una variable después de su inicialización. Es útil para límites y valores que deben permanecer constantes durante ese contexto.

```
final double LIMITE_TEMPERATURA = 80.0;
```

Intentar asignarle otro valor produce un error de compilación. Las constantes evitan números mágicos y expresan reglas mediante nombres.

```
final double LIMITE_TEMPERATURA = 80.0;
double temperatura = 76.5;

if (temperatura > LIMITE_TEMPERATURA) {
    System.out.println("Alerta");
}
```

8. Reglas y convenciones para nombres

Regla o convención	Ejemplo
Puede contener letras y números después del inicio	temperatura1
No comenzar con número	1temperatura ✗
No usar palabras reservadas	int class ✗
No usar espacios	cantidad piezas ✗
Usar camelCase	cantidadPiezas ✓
Constantes en MAYÚSCULAS	LIMITE_MAXIMO ✓

```
int cantidadPiezas = 500;
double temperaturaMotor = 72.5;
boolean maquinaActiva = true;
```

Java distingue mayúsculas y minúsculas. Nombres descriptivos hacen que el código sea más fácil de comprender y mantener.

9. Variables locales, parámetros y atributos

Una variable local se declara dentro de un método o bloque. Un parámetro recibe información cuando se invoca un método. Un atributo pertenece a una clase u objeto.

```
public static void mostrarTemperatura(double temperatura) {
    double limite = 80.0;
    System.out.println(temperatura);
}
```

`temperatura` es un parámetro y `limite` es una variable local. Ambas tienen un alcance limitado a ese método.

```
public class Maquina {
    String modelo;
    double temperatura;
}
```

`modelo` y `temperatura` son atributos. Este concepto conecta variables con programación orientada a objetos.

10. Variables y conversión de datos

A veces recibimos un dato con un tipo diferente al que necesitamos. Una conversión transforma un valor compatible a otro tipo.

```
String texto = "1500";  
int velocidad = Integer.parseInt(texto);
```

texto contiene caracteres. parseInt interpreta el texto como entero. Si el texto no representa un entero válido, puede producir una excepción. La conversión se estudiará con más profundidad posteriormente.

11. Ejemplos progresivos

Nivel 1 — Básico

```
String nombre = "Carlos";  
int edad = 30;  
  
System.out.println("Nombre: " + nombre);  
System.out.println("Edad: " + edad);
```

Se almacenan y muestran dos datos diferentes.

Nivel 2 — Intermedio

```
double precio = 125.50;  
int cantidad = 4;  
double total = precio * cantidad;  
  
System.out.println("Total: $" + total);
```

total depende de precio y cantidad; cambiar esos valores cambia el resultado.

Nivel 3 — Práctico industrial

```
String maquina = "Fresadora CNC";  
double temperatura = 74.5;  
int velocidad = 1600;  
int piezas = 555;  
boolean activa = true;  
  
System.out.println("Máquina: " + maquina);  
System.out.println("Temperatura: " + temperatura + " °C");  
System.out.println("Velocidad: " + velocidad + " RPM");  
System.out.println("Piezas: " + piezas);  
System.out.println("Activa: " + activa);
```

Cada variable representa una característica diferente de la máquina y el tipo refleja su naturaleza.

Nivel 4 — Profesional

```
public class Produccion {  
    static final int LIMITE_PIEZAS = 1000;  
  
    public static void main(String[] args) {  
        String linea = "Línea A";  
        int piezasBuenas = 940;  
        int piezasDefectuosas = 35;  
        int total = piezasBuenas + piezasDefectuosas;  
        double porcentajeBueno =  
            piezasBuenas * 100.0 / total;  
        boolean requiereRevision = total < LIMITE_PIEZAS;  
  
        System.out.println("Línea: " + linea);
```

```

System.out.println("Total: " + total);
System.out.println("Calidad: " + porcentajeBueno + "%");
System.out.println("Revisión: " + requiereRevision);
}
}

```

Aquí se combinan texto, enteros, decimales, booleanos y una constante. En un sistema real, estos valores podrían proceder de sensores, bases de datos o servicios.

12. Diagramas y relaciones

Etapa	Qué sucede
1. Declarar	tipo + nombre
2. Inicializar	primer valor
3. Leer	usar el valor
4. Modificar	asignar otro valor
5. Reutilizar	trabajar con el nuevo estado

Modelo: tipo → nombre → valor → uso → nuevo valor. Este ciclo aparece en producción, inventarios, videojuegos y aplicaciones empresariales.

13. Comparaciones importantes

Concepto	Qué es	Cuándo usarlo	Diferencia
int	Entero	Contadores y cantidades	No representa decimales
double	Decimal	Medidas/cálculos	Puede tener aproximación
boolean	Lógico	Estados/decisiones	Solo true/false
char	Carácter	Símbolo individual	Un carácter
String	Texto	Nombres/mensajes	Es una referencia
final	No reasignable	Constantes	No admite otra asignación

Errores de confusión comunes

No confundas char con String: 'A' es un carácter y "A" es un String. No confundas declaración con inicialización. No confundas una variable con su valor: piezas es el nombre y 500 es un posible valor. Tampoco confundas final con un valor global: final se refiere a la imposibilidad de reasignar.

14. Errores comunes al trabajar con Variables en Java

Usar una variable local sin inicializar

Se declara y se lee antes de asignarle valor. Java puede impedir la compilación. Corrige asignando un valor antes de usarla.

Elegir un tipo incorrecto

Se intenta almacenar un decimal en un entero o texto en un número. Elige el tipo correcto o realiza una conversión válida.

Confundir = con ==

= asigna y == compara en los contextos donde corresponde. Mezclarlos causa errores lógicos o de sintaxis.

Usar nombres poco claros

n, x o dato esconden el significado. Usa nombres como cantidadPiezas o temperaturaMotor.

Intentar cambiar una final

Una variable final no puede reasignarse después de inicializarse.

Confundir comillas

char usa comillas simples; String usa comillas dobles.

Ignorar precisión o rango

Un tipo puede no representar adecuadamente el rango o precisión requerida. Selecciona según el problema.

Usar números mágicos

Repetir límites como 80.0 dificulta el mantenimiento. Usa constantes descriptivas.

15. Buenas prácticas

- Usa nombres descriptivos y camelCase.
- Declara variables cerca de donde se utilizan cuando eso mejore la claridad.
- Elige el tipo por significado, rango y precisión.
- Inicializa variables locales antes de leerlas.
- Usa final para valores que no deben reasignarse.
- Evita variables globales innecesarias y controla el alcance.
- No guardes ni expongas información sensible sin considerar su protección.
- Evita conversiones innecesarias y comprende sus posibles pérdidas.
- Prefiere código legible frente a abreviaturas excesivas.
- En proyectos industriales documenta unidades como °C, RPM, volts y amperes.
- Usa nombres que indiquen claramente qué representa el dato.

16. Conectando conocimientos

Tema	Relación
Sintaxis	Una declaración de variable es una instrucción Java.
Salidas	Permite mostrar datos dinámicos.
Comentarios	Pueden explicar significado y unidades.
Operadores	Calculan y modifican valores.
Entrada	Los datos recibidos pueden almacenarse.

Tema	Relación
Decisiones	Las variables proporcionan condiciones.
Ciclos	Pueden actuar como contadores y estados.
Métodos	Parámetros y variables locales reutilizan lógica.
POO	Los atributos representan el estado de objetos.

17. Ejercicios prácticos

Ejercicio 1 — Comprensión básica

Problema: Declara variables para nombre, edad, ciudad y lenguaje favorito.

Requisitos: Usa tipos adecuados y muestra cada dato.

Pista: Pregunta qué tipo representa naturalmente cada información.

Ejercicio 2 — Aplicación

Problema: Crea un programa que calcule el total de una compra.

Requisitos: Usa precio, cantidad y total; el total debe depender de las otras variables.

Pista: No repitas el mismo valor dentro de varias expresiones.

Ejercicio 3 — Integración

Problema: Calcula producción con piezas buenas, defectuosas, total y porcentaje de calidad.

Requisitos: Combina variables, operadores, comentarios y salidas.

Pista: Separa datos originales de resultados calculados.

Ejercicio 4 — Caso práctico

Problema: Construye un registro de una máquina con modelo, temperatura, velocidad, piezas y estado.

Requisitos: Usa String, int, double, boolean y una constante de límite.

Pista: Decide primero qué tipo representa cada dato.

Ejercicio 5 — Reto

Problema: Diseña un panel de producción que administre línea, piezas, calidad, temperatura y revisión.

Requisitos: Incluye constantes, variables modificables y cálculos derivados.

Pista: Primero diseña los datos; después decide tipos y finalmente escribe el programa.

18. Mini proyecto del capítulo

Nombre: Panel de monitoreo de una máquina de producción

Contexto: una empresa necesita un programa inicial que almacene información de una máquina y genere un reporte. Los valores serán simulados mediante variables.

Objetivo: practicar declaración, inicialización, tipos, asignación, constantes, cálculos y uso de variables.

Requisitos funcionales: identificar máquina, temperatura, velocidad, piezas buenas/defectuosas, estado, total y porcentaje.

Requisitos técnicos: usar String, int, double, boolean y final; operadores y salidas; nombres descriptivos; comentarios para unidades y reglas.

Paso 1: diseña las variables. Para cada una escribe nombre, tipo, ejemplo de valor y motivo.

Paso 2: inicializa los datos y muestra un reporte.

Paso 3: agrega total y porcentaje de piezas buenas.

Paso final: agrega un límite constante y una condición de revisión. Revisa tipos y nombres.

Pistas: separa datos originales de resultados; usa nombres descriptivos; identifica qué valores cambian y cuáles deben permanecer constantes.

19. Comprueba tus conocimientos

1. Define una variable.
2. ¿Qué diferencia hay entre declarar e inicializar?
3. ¿Qué diferencia hay entre asignar y utilizar?
4. ¿Por qué Java utiliza tipos?
5. ¿Cuándo elegirías int en lugar de double?
6. ¿Qué representa boolean?
7. ¿Cuál es la diferencia entre char y String?
8. ¿Qué significa final?
9. ¿Qué ocurre en `int piezas; System.out.println(piezas);`?
10. ¿Qué tipo y valor tiene `double temperatura = 75.5;`?
11. ¿Qué ocurre después de `cantidad = 25` si antes valía 10?
12. ¿Por qué `cantidadPiezas` es mejor que `cp`?
13. ¿Qué es una variable local?
14. ¿Qué es un parámetro?
15. ¿Qué tipo usarías para saber si una máquina está activa?
16. ¿Por qué usar constantes para límites?
17. ¿Qué diferencia hay entre `char letra='A'` y `String letra="A"`?
18. ¿Cómo usarías variables en un inventario?

20. Respuestas y explicaciones

1. Elemento nombrado que representa un valor y tiene tipo y alcance.
2. Declarar indica tipo/nombre; inicializar asigna el primer valor.
3. Asignar coloca un valor; utilizar emplea el valor en una operación, salida o decisión.
4. Para definir valores válidos y detectar incompatibilidades.

5. Para enteros cuando no se necesita decimal y el rango es suficiente.
6. true o false.
7. char representa un carácter; String representa texto.
8. Que no puede recibir otra asignación después de inicializarse.
9. Una local puede no estar inicializada y Java impide leerla.
10. Es double y su valor inicial es 75.5.
11. La variable pasa a contener 25.
12. Comunica el propósito y facilita mantenimiento.
13. Variable declarada dentro de un método o bloque.
14. Dato recibido por un método al ser invocado.
15. boolean.
16. Expresan reglas con nombres y evitan números mágicos.
17. char usa comillas simples y un carácter; String usa comillas dobles.
18. Para almacenar código, descripción, cantidad, precio, mínimo y estado y utilizarlos en cálculos y decisiones.

21. Resumen del capítulo

Concepto	Para qué sirve	Cuándo usarlo	Error común
Variable	Representar datos	Cuando el programa necesita información	No inicializar
Tipo	Definir clase de dato	Al declarar	Tipo inadecuado
Asignación	Establecer/cambiar valor	Inicialización o actualización	Confundir = y ==
final	Evitar reasignación	Constantes/límites	Intentar cambiarla
Alcance	Controlar dónde se usa	Al declarar	Usarla fuera de contexto
Nombre	Comunicar propósito	Siempre	Nombre ambiguo

Ideas clave que debes recordar

- Una variable representa un dato mediante un nombre y un tipo.
- Declarar, inicializar y asignar son acciones diferentes.
- Java verifica muchos errores de tipos durante la compilación.
- int suele ser el entero general y double un decimal habitual.
- boolean representa estados verdadero/falso.
- char representa un carácter; String representa texto.
- final evita la reasignación.
- Los nombres claros mejoran la calidad profesional.
- El alcance determina desde dónde puede utilizarse.
- Las variables son base de operadores, decisiones, ciclos, métodos y objetos.

22. Mapa de aprendizaje

Conocimiento previo → Variables → Declaración e inicialización → Tipos → Asignación y modificación → Constantes → Alcance → Operadores y salidas → Decisiones y ciclos → Métodos y objetos → Proyectos profesionales.

Este capítulo prepara el camino para operadores y conversiones, estructuras de decisión y repetición, métodos, arreglos, clases y objetos.

Verificación final

Antes de avanzar, verifica que puedes responder sin consultar material externo: ¿qué es una variable?, ¿por qué existe?, ¿qué tipo elegirías para cada dato?, ¿cómo se declara e inicializa?, ¿cómo se modifica?, ¿qué significa final?, ¿qué es el alcance?, ¿cuáles son los errores comunes?, ¿cómo aplicarías variables a una máquina o inventario?